SYSTEM ARCHITECTURE DOCUMENT

Payroll-Integrated Real-Time Credit Infrastructure

Version 1.0

July 2026

Author

Clayton Soares da Mota

Database Architect — Financial Data Systems

Repository: github.com/claytonmota/payroll-credit-mvp

Live Deployment: payroll-credit.com

Document Control

Revision History

Version	Date	Author	Summary of Changes
1.0	2026-07-15	Clayton S. Mota	Initial formal release. Consolidates README, ROADMAP, and RFE_EVIDENCE into a single engineering artifact.

Distribution

- Petitioner (Clayton Soares da Mota) — primary author and technical owner
- Immigration counsel of record — for inclusion in I-140 NIW RFE response
- Public — via GitHub repository under Apache 2.0 license

Scope of this Document

This document is the authoritative reference for the technical design, implementation, and operational characteristics of the Payroll-Integrated Real-Time Credit Infrastructure prototype developed by Clayton Soares da Mota. It consolidates information previously distributed across the repository's README, ROADMAP, and RFE_EVIDENCE documents into a single engineering artifact suitable for formal review.

Related Documents

- Professional Plan submitted with I-140/EB-2 NIW petition (February 2026)
- RFE issued by USCIS Texas Service Center on June 22, 2026 (case IOE0936599919)
- docs/RFE_EVIDENCE.md — Reading guide mapping Professional Plan to implementation
- docs/DEMO.md — Reproducible end-to-end walkthrough
- docs/ROADMAP.md — Scope status and future work

Table of Contents

(Right-click the table above and choose "Update Field" to populate page numbers.)

1. Executive Summary

The Payroll-Integrated Real-Time Credit Infrastructure is a working reference implementation of a payroll-derived credit assessment platform, targeting the credit-invisible and thin-file segment of the United States consumer market. It was designed and implemented by Clayton Soares da Mota as the technical embodiment of the endeavor proposed in his I-140 EB-2 National Interest Waiver petition.

The system consumes normalized payroll events from producer sources (ADP, Workday, Gusto, Rippling, Paychex), computes a real-time income confidence score, aggregates the result with credit bureau data, and produces auditable eligibility decisions. All four microservices are deployed on Amazon Web Services (AWS EC2, us-east-1) and are reachable through HTTPS on payroll-credit.com.

1.1 Key Characteristics

- **Event-driven microservices architecture** spanning four independently deployable services.
- **Cross-language contracts** — three services in Java (Spring Boot 3.2) and one in C# (.NET 8), communicating exclusively via Apache Kafka events.
- **Polyglot persistence** — PostgreSQL for transactional state and audit records; MongoDB for aggregate profile documents.
- **Real-time income validation** — income confidence score computed deterministically from payroll history, independent of bureau lookup.
- **Auditable rules engine** — every eligibility decision persists a human-readable reasoning field, aligned with Basel II governance and CFPB adverse-action requirements.
- **Financial inclusion focus** — explicit THIN_FILE classification for consumers with payroll-verified income but no bureau history.
- **Production deployment** — AWS EC2 with Elastic IP, HTTPS-terminated subdomains, Docker Compose orchestration.

1.2 Alignment with the Professional Plan

Professional Plan Methodology	System Component
1 — Integrated Data Architecture and Interoperability	Ingestion Service, Kafka event back
2 — Cloud-Native Real-Time Risk Orchestration	Four microservices in Java (Spring E
3 — Advanced Data Governance and Financial Compliance	Immutable eligibility_decision audi rules engine (no black-box ML)

2. System Context

2.1 Problem Statement

The United States credit market suffers from two intersecting failures. First, according to the Consumer Financial Protection Bureau, approximately 26 million American adults are credit invisible — they have no record with the three main credit bureaus (Experian, Equifax, TransUnion). An additional 19 million have unscored credit files. Together, these 45 million consumers are systematically excluded from mainstream credit products despite many having stable, verifiable income.

Second, the bureau-first underwriting model is inherently latent. Credit reports reflect financial behavior with a delay of weeks to months. In an economy where employment can change overnight and where gig and contract work has grown to represent a substantial share of the workforce, historical bureau data is a poor proxy for current payment capacity.

This system's technical hypothesis is that real-time payroll data, when properly ingested, normalized, and analyzed, can serve as a first-class underwriting signal — either supplementing bureau data for thick-file consumers or standing alone for thin-file consumers who would otherwise be denied credit access.

2.2 System Boundary

The system boundary encompasses the ingestion, transformation, decisioning, and audit of payroll-derived credit signals. It explicitly does not include:

- Loan origination or servicing (assumed to be handled by the consuming lender)
- Regulatory reporting (produced downstream by the consuming institution)
- Fraud detection at the identity level (handled by upstream identity providers)
- Direct interaction with end consumers (no consumer-facing UI in this prototype)

2.3 Stakeholders and Consumers

Stakeholder	Interaction with the System
Payroll Provider	Publishes normalized pay-period events into the Ingestion Layer
Credit Union / Community Bank	Consumes eligibility decisions to underwrite thin-file members
Consumer-Lending Fintech	Consumes eligibility and credit profile data as an underwriting input
Regulatory Reviewer	Audits the eligibility_decision table for CFPB adverse-action reporting
Data Scientist / Underwriter	Queries the credit profile aggregate for portfolio analytics

3. Architecture Overview

3.1 Architectural Principles

The system is designed against a small set of explicit architectural principles. These principles govern trade-offs across the entire codebase and are the reference for any future extension.

Principle 1 — Event-Driven Communication

Services communicate exclusively via Kafka events. There are no synchronous service-to-service HTTP calls. This preserves independent deployability, absorbs load spikes through Kafka's broker buffer, and enables cross-language interoperability without shared libraries.

Principle 2 — Language-Agnostic Contracts

The Kafka event schema is the contract; the internal class name of a producing service is not. All producers disable the __TypeId__ header emission (see ADR-001). This means a Java service and a C# service can co-exist and consume the same event without any shared class hierarchy — as demonstrated by the credit-profile-service (C#) consuming events published by the income-verification-service (Java).

Principle 3 — Explainable Decisions

The eligibility rules engine is deterministic and rule-based. It does not use any black-box machine learning model. Every decision writes a human-readable reasoning field explaining the specific rules that fired. This supports CFPB adverse-action reporting (which requires a specific reason code to be provided to the consumer) and Basel II model risk management (which requires model outputs to be independently auditable).

Principle 4 — Immutable Audit

The eligibility_decision table is append-only. A decision is never updated once written; if the assessment changes, a new decision is inserted with a new UUID. This creates a complete audit trail that can be reconstructed at any point in time — a prerequisite for regulatory review.

Principle 5 — Polyglot Persistence

The system uses PostgreSQL for transactional state where relational guarantees matter (eligibility decisions, income verification results, raw payroll event records), and MongoDB for aggregate documents whose shape evolves over time (credit profiles combining income and bureau data). Each service owns its own database schema.

3.2 High-Level Architecture Diagram

The architecture is visualized in the diagram maintained in the repository at docs/architecture-v4.png. The diagram is reproduced logically in Section 4 through detailed descriptions of each layer.

> Reviewer note: the .excalidraw source file and the rendered PNG are included in the repository. The PNG should be attached to any review package alongside this document for a complete visual reference.

3.3 Deployment Topology

The system is deployed as a single-host Docker Compose stack on an AWS EC2 instance in region us-east-1. This is intentional for the prototype phase: it maximizes ease of reproduction and inspection while validating the design end-to-end. Production topology recommendations for multi-node deployment are discussed in Section 10 (Non-Functional Characteristics) and Section 11 (Roadmap).

Component	Value
Cloud Provider	Amazon Web Services (AWS)
Region	us-east-1 (N. Virginia)
Instance Type	m7i-flex.large (2 vCPU, 8 GB RAM)
Operating System	Ubuntu 24.04 LTS
Public IP	Elastic IP 3.229.114.98
Domain	payroll-credit.com (four subdomains, HTTPS)
Container Runtime	Docker Engine 26+, Docker Compose v2
TLS Certificates	Let's Encrypt (auto-renewal via certbot)

4. Logical Architecture — Layer by Layer

The system is organized into ten logical layers. Each layer has a single, well-defined responsibility. This section describes each layer's purpose, components, and connection to the codebase.

4.1 Data Sources Layer

External systems that supply the raw data consumed by the platform. In the prototype, these are simulated by the DEMO walkthrough; in production, real integrations would be implemented in the Ingestion Layer's connector adapters.

Source Category	Examples
Payroll Providers	ADP, Workday, Paychex, Gusto, Rippling
Employer / HR Systems	REST, SOAP, SFTP, Webhooks, custom APIs
Open Banking / Account Aggregators	Plaid, MX
Credit Bureaus	Experian, Equifax, TransUnion, FICO
Third-Party Data Providers	Fraud signals, device intelligence, identity verification, AML/KYC

4.2 Ingestion Layer

Receives payroll events from external sources, normalizes them into the canonical PayrollEvent contract, and publishes them onto the payroll.events Kafka topic. In this prototype the ingestion is implemented as a single REST endpoint that accepts an already-normalized event; in production, the layer would include an API gateway (Kong or Apigee), per-provider connector adapters, a file/SFTP processor for batch feeds, and a webhook receiver.

Implementation

- Service: ingestion-service (Java 17, Spring Boot 3.2)
- Endpoint: POST https://ingestion.payroll-credit.com/v1/payroll/events
- Kafka topic: payroll.events (produces)
- Persistence: none (stateless)

4.3 Event Streaming and Integration Layer

Apache Kafka is the backbone of the system's event flow. All inter-service communication happens through Kafka topics; no service calls another service directly over HTTP or gRPC. This provides three properties simultaneously: independent scaling, buffered load, and language-agnostic contracts.

Topics

Topic	Producer	Consumers
payroll.events	ingestion-service	income-verification-service

Topic	Producer	Consumers
income.verified	income-verification-service	decision-service, credit-profi

4.4 Microservices Layer

Four domain-oriented microservices implement the business logic of the platform. Each service is independently deployable, has its own persistence, and communicates only via Kafka events with the others.

4.4.1 Ingestion Service

- Language: Java 17 (Spring Boot 3.2)
- Responsibility: normalize and publish payroll events
- Public endpoint: POST /v1/payroll/events
- Kafka: produces to payroll.events

4.4.2 Income Verification Service

- Language: Java 17 (Spring Boot 3.2)
- Responsibility: compute income confidence score from payroll history
- Public endpoint: GET /v1/income/verification/{userId}
- Kafka: consumes payroll.events; produces to income.verified
- Persistence: PostgreSQL — payroll_event_record, income_verification_result

4.4.3 Decision Service

- Language: Java 17 (Spring Boot 3.2)
- Responsibility: apply eligibility rules and emit eligibility decision
- Public endpoint: GET /v1/eligibility/{userId}
- Kafka: consumes income.verified
- Persistence: PostgreSQL — eligibility_decision (append-only audit)

4.4.4 Credit Profile Service

- Language: C# .NET 8 (ASP.NET Core)
- Responsibility: aggregate income + bureau data into a document per user
- Public endpoint: GET /v1/credit-profile/{userId}
- Kafka: consumes income.verified
- Persistence: MongoDB — credit_profiles collection

4.5 API Gateway Layer (External)

In this prototype, TLS termination and routing are handled by an nginx reverse proxy running on the same EC2 host, with subdomain-based virtual hosts. A production deployment would replace this with a managed gateway (AWS API Gateway, Kong, or Apigee) providing rate limiting, authentication, throttling, and centralized monitoring.

4.6 Consumers Layer

External systems and users that consume the platform's outputs. In this prototype, no real consumer is integrated; the DEMO walkthrough simulates consumer queries via curl.

- Lenders and banks (credit unions, regional banks, digital banks)
- Fintechs (personal loans, BNPL, auto loans, workforce lending)
- Internal tools (underwriter portal, risk dashboard, admin console)

4.7 Data Layer

Two operational data stores back the running services. The choice of engine per service reflects the shape of the data (relational for audit records; document for evolving aggregates) rather than a top-down architectural mandate.

Store	Type	Owning Service	Purp
PostgreSQL	Relational	income-verification, decision	Tran
MongoDB	Document	credit-profile	Aggr

4.8 Cross-Cutting Services

- Configuration — Spring Boot's application.yml (Java) and appsettings.json (C#)
- Logging — SLF4J with Logback (Java) and Microsoft.Extensions.Logging (C#)
- Health checks — Spring Actuator (/actuator/health) and ASP.NET Core health endpoints
- Build and packaging — Maven (Java) and dotnet CLI (C#), containerized via multi-stage Dockerfiles

4.9 Security and Compliance Layer

The security posture of the prototype focuses on transport-level protection and audit integrity. Deeper controls (encryption at rest, IAM/RBAC, WAF/DDoS protection) are documented as production requirements in the roadmap.

- Transport encryption — HTTPS via Let's Encrypt on all four subdomains
- Audit integrity — append-only eligibility_decision table
- Compliance alignment — SOC 2, PCI-DSS, CFPB (design-time consideration)
- Data privacy — CCPA, GLBA (Gramm-Leach-Bliley Act)
- Basel II — model transparency via deterministic rules and reasoning field

4.10 End-to-End Flow

A single payroll event traverses the platform in the following steps:

1) User (or provider system) POSTs a normalized PayrollEvent to the Ingestion Service.

2) Ingestion Service publishes the event to the payroll.events Kafka topic.

3) Income Verification Service consumes the event, persists the raw record, recomputes the income confidence score across the user's history, and produces an IncomeVerifiedEvent to the income.verified topic.

4) Decision Service consumes the IncomeVerifiedEvent, applies its rules engine, and persists an EligibilityDecision with human-readable reasoning.

5) Credit Profile Service consumes the same IncomeVerifiedEvent (independent consumer group), performs a bureau lookup, updates the aggregate CreditProfile document in MongoDB, and classifies the file (THIN_FILE / STANDARD / RICH_FILE).

6) Consumer systems query the decision-service and credit-profile-service REST endpoints for the latest results.

5. Microservices — Detailed Specification

5.1 Ingestion Service

Responsibility

Receive external payroll events, validate their structure, and publish them to Kafka. In the prototype, the service accepts already-normalized events; in production, this layer would include per-provider connector adapters.

Runtime

Property	Value
Language	Java 17
Framework	Spring Boot 3.2
Container port	8081
Public endpoint	https://ingestion.payroll-credit.com
Persistence	None (stateless)
Kafka role	Producer to payroll.events

Key Classes

- IngestionServiceApplication — Spring Boot entry point
- PayrollEventController — REST controller for the /v1/payroll/events endpoint
- PayrollEventProducer — thin wrapper over KafkaTemplate that publishes to payroll.events
- KafkaProducerConfig — configures JsonSerializer with ADD_TYPE_INFO_HEADERS = false (see ADR-001)
- PayrollEvent — DTO for the canonical event contract

5.2 Income Verification Service

Responsibility

Consume payroll events, persist them for historical analysis, and compute a real-time income confidence score across a rolling window of up to twelve pay periods per user. The score is the platform's substitute for static credit history.

Confidence Score Algorithm

The algorithm is deliberately transparent, deterministic, and testable:

1) Each pay event's net pay is normalized to a monthly-equivalent value based on its pay frequency (weekly, biweekly, semimonthly, monthly).

2) The coefficient of variation (standard deviation divided by mean) is computed across the last N events, where N is bounded by 12.

3) The confidence score is defined as 1 minus the coefficient of variation, clamped to the interval [0, 1]. Perfectly stable income yields 1.0; highly variable income approaches 0.

4) A stability label is derived from thresholds on the score: STABLE (≥ 0.85), MODERATE (≥ 0.60), VOLATILE (< 0.60), or INSUFFICIENT_DATA (fewer than 2 events).

Runtime

Property	Value
Language	Java 17
Framework	Spring Boot 3.2 + Spring Data JPA
Container port	8082
Public endpoint	https://income.payroll-credit.com
Database	PostgreSQL (owns: incomeverification database)
Kafka role	Consumer of payroll.events; producer to income.verified

5.3 Decision Service

Responsibility

Consume IncomeVerifiedEvent messages and apply the eligibility rules engine to produce an auditable EligibilityDecision. The rules engine is deterministic and rule-based — not a machine learning model — which supports regulatory transparency requirements.

Rules Engine — Decision Path

The engine evaluates the following rules in order and stops at the first match:

1) INSUFFICIENT_DATA (fewer than 3 pay events considered) → REVIEW.

2) Average monthly income below the configured minimum ($1,800) → DENIED.

3) Confidence score ≥ 0.75 AND stability label STABLE → APPROVED.

4) Confidence score ≥ 0.50 (moderate) → REVIEW.

5) Otherwise → DENIED.

Credit limit sizing for APPROVED decisions is 30% of average monthly income at APR 18.99% for STABLE income (22.99% for MODERATE). REVIEW decisions receive a conservative preliminary limit of 10% at APR 24.99%. DENIED decisions receive $0.

Runtime

Property	Value
Language	Java 17
Framework	Spring Boot 3.2 + Spring Data JPA
Container port	8083

Property	Value
Public endpoint	https://decision.payroll-credit.com
Database	PostgreSQL (owns: decisions database)
Kafka role	Consumer of income.verified

5.4 Credit Profile Service

Responsibility

Aggregate the real-time income signals with a credit bureau lookup into a single document per user, classifying the file as THIN_FILE, STANDARD, or RICH_FILE. This service demonstrates the cross-language capability of the architecture — it is written in C# .NET 8 and consumes the same Kafka topic as the Java-based Decision Service without any shared library.

Classification Rules

- THIN_FILE — bureau returned no history (or bureau lookup indicates "no file")
- RICH_FILE — bureau score ≥ 720 AND income label = STABLE
- STANDARD — everything else

Bureau Lookup Adapter

The prototype uses a deterministic stub (DeterministicBureauLookupStub) that produces reproducible results based on a hash of the userId. userIds ending in -thinfile always return no bureau history, allowing the THIN_FILE case to be demonstrated on demand. A production adapter would replace this stub with an HTTP client against Experian, Equifax, or TransUnion sandbox APIs.

Runtime

Property	Value
Language	C# (.NET 8)
Framework	ASP.NET Core minimal-host, Confluent.Kafka, MongoDB.Driver
Container port	8084
Public endpoint	https://creditprofile.payroll-credit.com
Database	MongoDB (owns: creditprofile database)
Kafka role	Consumer of income.verified

6. Data Model

6.1 PostgreSQL — Transactional Store

PostgreSQL holds two logical databases, each owned by a distinct service.

6.1.1 incomeverification database

payroll_event_record

Historical record of every payroll event received for a user. Retained so that confidence-score calculations can operate on a rolling window rather than a single data point.

Column	Type	Notes
id	BIGSERIAL PRIMARY KEY	Surrogate key
user_id	VARCHAR NOT NULL	External user identifier
employer_name	VARCHAR	
pay_period_start	DATE NOT NULL	
pay_period_end	DATE NOT NULL	
gross_pay	DOUBLE PRECISION NOT NULL	
net_pay	DOUBLE PRECISION NOT NULL	
pay_frequency	VARCHAR	WEEKLY, BIWEEKLY, SEMIMO
source_provider	VARCHAR	e.g., "ADP", "Gusto"
received_at	TIMESTAMP NOT NULL	Server-side ingest time

income_verification_result

One row per user representing the current computed income assessment. Updated (not appended) as new events arrive.

Column	Type	Notes
user_id	VARCHAR PRIMARY KEY	
average_monthly_income	DOUBLE PRECISION NOT NULL	
income_confidence_score	DOUBLE PRECISION NOT NULL	[0, 1]
income_stability_label	VARCHAR NOT NULL	STABLE, MODER
pay_events_considered	INTEGER NOT NULL	
last_updated	TIMESTAMP NOT NULL	

6.1.2 decisions database

eligibility_decision

Immutable audit record of every eligibility decision. One row per decision — a new decision inserts a new row rather than updating an existing one.

Column	Type	Notes
decision_id	VARCHAR(64) PRIMARY KEY	UUID
user_id	VARCHAR NOT NULL	
decision	VARCHAR(32) NOT NULL	APPROVED, REV
credit_limit_usd	DOUBLE PRECISION NOT NULL	
suggested_apr	DOUBLE PRECISION NOT NULL	
average_monthly_income	DOUBLE PRECISION NOT NULL	
income_confidence_score	DOUBLE PRECISION NOT NULL	
income_stability_label	VARCHAR NOT NULL	
reasoning	VARCHAR(2000) NOT NULL	Human-readable
decided_at	TIMESTAMP NOT NULL	

6.2 MongoDB — Aggregate Store

credit_profiles collection

One document per user representing the aggregate credit profile. Chosen as a document store because the profile combines heterogeneous data (income, bureau, historical snapshots, classification) whose shape is expected to evolve as new signal categories are added over time.

Document schema (JSON):

```
{
  "_id": "user-1001",
  "averageMonthlyIncome": 5306.7,
  "incomeConfidenceScore": 1.0,
  "incomeStabilityLabel": "STABLE",
  "payEventsConsidered": 4,
  "bureauScore": 712,
  "bureauSource": "Experian",
  "thinFileClassification": "STANDARD",
  "incomeHistory": [
    {
      "averageMonthlyIncome": 5306.7,
      "incomeConfidenceScore": 1.0,
      "incomeStabilityLabel": "STABLE",
      "payEventsConsidered": 4,
      "observedAt": "2026-07-13T22:54:03Z"
    }
  ],
  "lastUpdated": "2026-07-13T22:54:03Z",
  "createdAt": "2026-07-13T22:54:03Z"
```

```
}
```

6.3 Kafka — Event Schemas

6.3.1 payroll.events

Published by ingestion-service. Consumed by income-verification-service.

```
{
  "userId": "user-1001",
  "employerName": "PortMiami Logistics LLC",
  "payPeriodStart": "2026-06-01",
  "payPeriodEnd": "2026-06-15",
  "grossPay": 3200.00,
  "netPay": 2450.00,
  "payFrequency": "BIWEEKLY",
  "sourceProvider": "Gusto"
}
```

6.3.2 income.verified

Published by income-verification-service. Consumed by decision-service and credit-profile-service (independent consumer groups).

```
{
  "userId": "user-1001",
  "averageMonthlyIncome": 5306.7,
  "incomeConfidenceScore": 1.0,
  "incomeStabilityLabel": "STABLE",
  "payEventsConsidered": 4,
  "verifiedAt": "2026-07-13T22:54:03Z"
}
```

7. API Reference

All public APIs are served over HTTPS on subdomains of payroll-credit.com. All endpoints accept and return application/json. Health endpoints are always available and are used for external monitoring.

7.1 Ingestion Service

POST /v1/payroll/events

Accepts a normalized payroll event. Returns 202 Accepted immediately after successful publication to Kafka.

Request body: PayrollEvent (see Section 6.3.1)

Success response (202):

```
{
  "status": "accepted",
  "userId": "user-1001",
  "message": "Payroll event queued for income verification"
}
```

GET /v1/payroll/health

Returns 200 OK if the service is running.

7.2 Income Verification Service

GET /v1/income/verification/{userId}

Returns the latest computed income verification result for the given user. Returns 404 if no events have been processed for this user yet.

Success response (200):

```
{
  "userId": "user-1001",
  "averageMonthlyIncome": 5306.7,
  "incomeConfidenceScore": 1.0,
  "incomeStabilityLabel": "STABLE",
  "payEventsConsidered": 4,
  "lastUpdated": "2026-07-13T22:54:03Z"
}
```

7.3 Decision Service

GET /v1/eligibility/{userId}

Returns the most recent eligibility decision for the given user. Returns 404 if no decision exists yet.

Success response (200):

```
{
  "decisionId": "3f2c...uuid...",
  "userId": "user-1001",
  "decision": "APPROVED",
  "creditLimitUsd": 1592.01,
  "suggestedApr": 18.99,
  "averageMonthlyIncome": 5306.7,
  "incomeConfidenceScore": 1.0,
  "incomeStabilityLabel": "STABLE",
```

```
  "reasoning": "Stable income stream with high confidence score (1.0). Average
monthly income meets threshold.",
  "decidedAt": "2026-07-13T22:54:03Z"
}
```

GET /v1/eligibility/{userId}/history

Returns the complete list of decisions ever made for the given user, ordered by decidedAt descending. Supports full audit trail retrieval.

7.4 Credit Profile Service

GET /v1/credit-profile/{userId}

Returns the aggregate credit profile document for the given user. Returns 404 if no events have been processed.

Success response (200): CreditProfile document (see Section 6.2)

7.5 Formal specifications

The endpoints described above are formally specified as OpenAPI 3.0.3 documents, one per service, held in docs/api/ in the repository. Each document is self-contained and can be opened in any OpenAPI viewer, imported into Postman or Insomnia, or used to generate a client library in any supported language.

Service	Specification file
Ingestion	docs/api/ingestion-service.openapi.yaml
Income Verification	docs/api/income-verification-service.openapi.yaml
Decision	docs/api/decision-service.openapi.yaml
Credit Profile	docs/api/credit-profile-service.openapi.yaml

Beyond request and response shapes, these documents record the decision rules a consuming institution would need in order to integrate: the pay-frequency multipliers used for monthly normalization, the threshold table mapping a confidence score to a stability label, the ordered rules the eligibility engine evaluates, the credit limit and APR sizing policy, and the thin-file classification conditions. They are contracts rather than documentation — a third party could integrate against them without reading the source.

> The specifications in this release are maintained by hand. The procedure for generating them from source annotations instead — via springdoc-openapi for the Spring Boot services and Swashbuckle for the .NET service, which would additionally expose an interactive Swagger UI on each public subdomain — is documented in docs/api/ENABLING_LIVE_API_DOCS.md.

8. Technology Stack

8.1 Runtime Stack

Category	Selection	Rationale
Java runtime	JDK 17 (Temurin)	Long-term support; native re
Java framework	Spring Boot 3.2	Industry standard for JVM mi
.NET runtime	.NET 8 LTS	Long-term support; latest AS
.NET libraries	ASP.NET Core, Confluent.Kafka 2.4, MongoDB.Driver 2.25	First-party or reference imple
Event broker	Apache Kafka (Confluent images 7.5)	De facto standard for high-th
Relational database	PostgreSQL 16	Open source, standards-com
Document store	MongoDB 7	Best-known document datab
Container runtime	Docker Engine 26+	Ubiquitous; supports multi-st
Orchestration	Docker Compose v2	Sufficient for single-host prot
Operating system	Ubuntu 24.04 LTS	Long-term support; broad to

8.2 Development and Build

- Version control — Git, hosted on GitHub
- Build tools — Maven 3.9 (Java), dotnet CLI (C#)
- Package management — npm (only for docs tooling); no runtime JavaScript
- Testing frameworks — JUnit 5 with Mockito (Java); xUnit with Moq (C#)
- Container images — Multi-stage Dockerfiles targeting Alpine-based JREs and mcr.microsoft.com/dotnet/aspnet:8.0

8.3 Alignment with the Professional Plan

The Professional Plan submitted with the original I-140 petition specified Java (Spring Boot), C# (.NET), PostgreSQL, and MongoDB as the target implementation stack. This prototype uses that exact stack. The Plan also mentioned SQL Server and DynamoDB as potential complementary stores; those are documented in the roadmap as candidates for future services (see Section 11).

9. Architecture Decision Records

Architecture Decision Records (ADRs) capture non-obvious design choices along with the context and rationale that produced them. They exist to prevent silent drift and to give future maintainers or reviewers the reasoning behind observable code.

9.1 ADR-001 — Producers do not emit Kafka type headers

Status

Accepted.

Context

Spring Kafka's default JsonSerializer emits a __TypeId__ header on every published message. The header contains the fully qualified class name of the producer's DTO. When a consumer with JsonDeserializer receives such a message, it looks up that class name against a configured trusted-packages list. If the consuming service uses a different package (as inevitably happens across services and languages), deserialization fails.

Decision

All Kafka producers in this system disable type-info headers via ADD_TYPE_INFO_HEADERS = false. The Kafka event schema — the JSON structure — is the contract; the producer's internal class name is an implementation detail.

Consequences

- Positive: services remain independently deployable across languages. The C# credit-profile-service consumes the same income.verified topic as the Java decision-service without any shared library.
- Positive: future services in Python, Go, or Rust can consume existing topics with no changes to the producers.
- Trade-off: consumers must know the target DTO type at compile time — there is no automatic type resolution from headers. This is acceptable and, arguably, desirable: it forces explicit contract awareness.

References

- ingestion-service/src/main/java/com/mota/ingestion/config/KafkaProducerConfig.java
- income-verification-service/src/main/java/com/mota/incomeverification/config/KafkaConfig.java (see incomeVerifiedProducerFactory)

9.2 ADR-002 — Deterministic rules engine over machine learning

Status

Accepted.

Context

The eligibility decision could be produced by a black-box ML classifier trained on historical outcomes, by a deterministic rules engine, or by a hybrid. ML approaches typically achieve higher predictive accuracy at the cost of interpretability.

Decision

The decision-service implements a deterministic, rule-based engine (EligibilityRulesEngine) that produces a categorical outcome and a human-readable reasoning string. No machine learning is used in the decision path.

Consequences

- CFPB adverse-action reporting requires a specific reason code to be delivered to the consumer. The reasoning field is directly consumable for this purpose.
- Basel II model risk management requires model outputs to be independently reproducible. The engine is deterministic and unit-tested.
- Trade-off: predictive accuracy may be lower than an equivalent ML model. Future iterations may introduce ML at the feature-engineering layer (assisting the rules) rather than replacing them.

10. Non-Functional Characteristics

10.1 Availability

The prototype runs on a single AWS EC2 instance and is not designed for high availability. Any hardware failure, host restart, or maintenance event results in downtime. This is acceptable for demonstration purposes. A production topology would distribute services across multiple instances, deploy Kafka as a multi-broker cluster, and use managed databases (RDS, DocumentDB or MongoDB Atlas) with automated failover.

10.2 Scalability

The event-driven architecture is horizontally scalable by consumer group. Each microservice can be scaled independently by adding replicas that join the same Kafka consumer group, at which point Kafka rebalances partitions across the replicas.

Formal load testing against the production deployment is planned as a follow-up deliverable; expected result on the current single-instance topology is on the order of hundreds of decisions per second, with latency dominated by Postgres write times for the audit record.

10.3 Performance

The end-to-end latency from POST /v1/payroll/events to eligibility_decision persistence is observed at under one second in normal operation. The dominant contributors are: network round-trip to Kafka (typically 5-20ms per hop), consumer poll cycle (up to 500ms depending on configuration), and Postgres commit (typically 5-15ms).

10.4 Auditability

Every eligibility decision writes a full audit record to eligibility_decision including input signals, output decision, reasoning, and timestamp. The table is append-only. A SELECT query with a time range reconstructs the state of the platform at any point in the past.

10.5 Security

- Transport — HTTPS via Let's Encrypt on all public subdomains; certificates auto-renew via certbot
- Storage — Postgres and MongoDB are not exposed to the public internet; only the four microservice ports (8081-8084) are open on the Security Group
- Compute — Docker containers do not run as root; secrets are provided via environment variables at container start
- Roadmap — IAM/RBAC, encryption at rest, WAF/DDoS, and secrets manager integration are documented as production requirements

10.6 Observability

- Structured logs via SLF4J (Java) and Microsoft.Extensions.Logging (C#), currently written to container stdout
- Health endpoints on all services (/v1/[svc]/health)
- Spring Actuator on Java services for JVM and application metrics
- Roadmap — full observability stack (Prometheus, Jaeger, ELK) planned for production topology

11. Roadmap and Known Gaps

This section documents, with full transparency, the architectural components that are planned but not yet implemented. Their absence does not represent a technical limitation of the approach — the same engineering patterns applied in the four existing services extend directly to each of these.

11.1 Planned Services

Component	Purpose
Identity Service	OAuth 2.0 / OIDC authentication, consent management
Employment Service	HR feed integration, employer verification, tenure scoring
Account Aggregation Service	Plaid / MX integration for bank account and cash flow data
Affordability Service	DTI and expense analysis using bank transaction data
Notification Service	Email, SMS, and push notification pipeline
Real HTTP Bureau Adapter	Replace deterministic stub with Experian/Equifax/TransUnion sandbox i

11.2 Infrastructure Enhancements

- External API Gateway — Kong or Apigee with JWT verification, rate limiting, and centralized routing
- SQL Server as a complementary operational store — mentioned in the Professional Plan
- DynamoDB as a complementary NoSQL store — mentioned in the Professional Plan
- Data Warehouse export — periodic ETL of eligibility_decision into Snowflake or Redshift for analytics
- Full observability stack — Prometheus metrics, Jaeger distributed tracing, ELK log aggregation
- CI/CD pipeline — GitHub Actions workflows for automated build, test, and deployment
- Multi-node production topology — split services across EC2 instances behind an Application Load Balancer, with Kafka cluster and managed RDS/DocumentDB

12. Reproducibility

The system is fully reproducible from source. Any reviewer with Docker installed can clone the repository and bring the stack up on their own machine in fewer than fifteen minutes.

12.1 Local Reproduction

```
git clone https://github.com/claytonmota/payroll-credit-mvp
cd payroll-credit-mvp
docker compose up --build

# Wait ~2 minutes for services to become healthy, then follow docs/DEMO.md
```

12.2 Live Reproduction

Reviewers can also validate the deployed system directly without cloning. The following sequence sends a payroll event, waits for the pipeline to process, and queries each downstream service:

```
# Send four biweekly payroll events for a demo user
for i in 1 2 3 4; do
  curl -X POST https://ingestion.payroll-credit.com/v1/payroll/events \
    -H "Content-Type: application/json" \
    -d '{"userId":"reviewer-001","employerName":"Reviewer
Test","payPeriodStart":"2026-06-01","payPeriodEnd":"2026-06-
15","grossPay":3200.00,"netPay":2450.00,"payFrequency":"BIWEEKLY","sourceProvid
er":"Gusto"}'
done

# Wait ~10 seconds, then query each service:
curl https://income.payroll-credit.com/v1/income/verification/reviewer-001
curl https://decision.payroll-credit.com/v1/eligibility/reviewer-001
curl https://creditprofile.payroll-credit.com/v1/credit-profile/reviewer-001
```

Reviewer note: the reproducibility above is a design goal. If any endpoint returns an unexpected error, the responsible party is the author (Clayton Mota), who can be reached through the GitHub repository or through immigration counsel of record.

Appendix A — Glossary

APR — Annual Percentage Rate — the annual cost of credit expressed as a percentage

Bureau — Consumer credit bureau (Experian, Equifax, TransUnion, FICO)

CFPB — Consumer Financial Protection Bureau (US federal regulator)

Credit Invisible — A consumer with no record at any consumer credit bureau

DTI — Debt-to-Income ratio, a standard underwriting affordability metric

ETL — Extract, Transform, Load — batch data movement pattern

EWA — Earned Wage Access — early access to earned but unpaid wages

Kafka — Apache Kafka, distributed event streaming platform

MVP — Minimum Viable Product — smallest implementation that demonstrates a design

OIDC — OpenID Connect — identity layer on top of OAuth 2.0

p50/p95/p99 — Percentiles (median, 95th, 99th) of a latency distribution

PPA — Provisional Patent Application (US Patent and Trademark Office)

RFE — Request for Evidence (US Citizenship and Immigration Services)

Thin File — A consumer with limited credit bureau history

Appendix B — References

- Consumer Financial Protection Bureau. Data Point: Credit Invisibles. Various reports.
- Basel Committee on Banking Supervision. Basel II framework.
- Consumer Financial Protection Bureau. Regulation B (Equal Credit Opportunity Act) — adverse action notices.
- Apache Kafka documentation. https://kafka.apache.org/documentation/
- Spring Kafka reference. https://docs.spring.io/spring-kafka/reference/
- Matter of Dhanasar, 26 I&N Dec. 884 (AAO 2016).

— End of Document —